

MODULE 18 · PART 6 - SETS AND GROUPING

Set Operations

UNION, INTERSECT, EXCEPT. The one place in SQL where two nulls are equal.

PART 6 of 8 MODULE 18 of 26 RUNS about 26 min

Why these get a module

- ◆ **They are the first thing anybody learns after a join**, and the last thing anybody reads the rules for.
- ◆ **The syntax is so simple that nobody goes back**, so four surprising behaviours survive for entire careers.
- ◆ **And one of them is genuinely useful**, because null equality makes EXCEPT the best diff tool SQL has.
- ◆ **Which part seven depends on**. Every regression test in this course is two EXCEPTs.

THE TAKEAWAY

Four behaviours: positional matching, null equality, duplicate removal, and precedence. Three are hazards and one is the reason to reach for these at all.

A join compares rows **side by side**. A set operation compares them **as whole rows**.

The three operations, and their ALL forms

OPERATION	WHAT IT RETURNS
UNION	Rows in either, with duplicates removed across both.
UNION ALL	Rows in either, everything kept. No comparison at all.
INTERSECT	Distinct rows present in both.
INTERSECT ALL	Rows in both, kept $\min(a, b)$ times each.
EXCEPT	Distinct rows in the first and not the second.
EXCEPT ALL	Rows in the first, minus one occurrence per match in the second.

THE TAKEAWAY

The plain forms work on sets and the ALL forms work on multisets. That distinction only shows up when a row appears more than once, which is exactly when it matters.

Reading a set operation out loud

WRITTEN	SAID
<code>A UNION ALL B</code>	every row of A, then every row of B, unchanged
<code>A UNION B</code>	...and remove duplicates, at the cost of a sort
<code>A INTERSECT B</code>	the distinct rows that appear in both
<code>A EXCEPT B</code>	the distinct rows of A that do not appear in B
<code>A EXCEPT ALL B</code>	each row of A, minus one copy per match in B

THE TAKEAWAY

Row two's second clause is the whole cost of the keyword, and it is not in the syntax. Saying it out loud is what makes you ask whether you need it.

All six, on eight rows

```

1 a = (1,'x') (1,'x') (2,'y') (3,NULL) (4,NULL)
2 b = (1,'x') (2,'y') (2,'y') (3,NULL)
3
4 UNION          4 (1,x) (2,y) (3,NULL) (4,NULL)
5 UNION ALL      9 everything, both sides
6 INTERSECT      3 (1,x) (2,y) (3,NULL)
7 INTERSECT ALL  3 same here: min(2,1) and min(1,2) are both 1
8 EXCEPT       1 (4,NULL)
9 EXCEPT ALL   2 (1,x) (4,NULL)

```

8 EXCEPT removes ALL copies of (1,x), because b contains one. EXCEPT ALL removes one, leaving one.

6 And (3,NULL) survives INTERSECT. Two nulls matched, which no join would have done.

THE TAKEAWAY

Measured. Nine rows of input, six different answers, and two of them differ only in how they count duplicates.

In a set operation, **NULL equals NULL**. It is the only place in SQL where that is true.

The same two rows, two answers

```
1 a has (3, NULL). b has (3, NULL).
2
3 SELECT * FROM a INTERSECT SELECT * FROM b
4     -> (3, NULL) is returned
5
6 SELECT * FROM a JOIN b USING (k, v) WHERE k = 3
7     -> 0 rows
8
9 -- identical data, opposite answers, and neither is wrong
```

7 The join compares NULL with NULL, which is UNKNOWN, which is not TRUE. So no match.

THE TAKEAWAY

Measured on the same two rows. The set operation says they are the same row. The join says it cannot tell. Both are following their own rules correctly.

Three ways to say 'not in the other table'

```
1 src = (1) (2) (3) (4)      ex = (1) (2) (NULL)
2
3 SELECT * FROM src WHERE k NOT IN (SELECT k FROM ex)
4     -> 0 rows
5
6 SELECT * FROM src s
7     WHERE NOT EXISTS (SELECT 1 FROM ex e WHERE e.k = s.k)
8     -> 2 rows   (3) (4)
9
10 SELECT k FROM src EXCEPT SELECT k FROM ex
11     -> 2 rows   (3) (4)
```

4 Module two's trap, measured. One null in the exclusion list and the answer is nothing at all.

THE TAKEAWAY

Three phrasings of one question. NOT EXISTS and EXCEPT are null-safe by different mechanisms, and NOT IN is not null-safe by any.

Which of the three to reach for

- ◆ **NOT EXISTS** when you need other columns from the source row, because it is a filter and keeps the row intact.
- ◆ **EXCEPT** when comparing whole result sets, especially across many columns, because it handles all of them at once.
- ◆ **NOT IN** essentially never, because it is unsafe on any nullable column and offers nothing the other two lack.
- ◆ **And EXCEPT deduplicates**, which NOT EXISTS does not. If a source row appears twice, EXCEPT returns it once.

THE TAKEAWAY

The last point is the real difference between them. EXCEPT is a set operation and returns a set; NOT EXISTS is a filter and returns whatever survived it.

Eight rows, and the answers by hand

```

1 a: (1,x) (1,x) (2,y) (3,NULL) (4,NULL)
2 b: (1,x)      (2,y) (2,y)      (3,NULL)
3
4 row      in a  in b    UNION  INT  INT ALL  EXC  EXC ALL
5 (1,x)    2    1      1      1    1    1    0    1
6 (2,y)    1    2      1      1    1    1    0    0
7 (3,NULL) 1    1      1      1    1    1    0    0
8 (4,NULL) 1    0      1      0    0    0    1    1

```

5 EXCEPT gives 0 and EXCEPT ALL gives 1, because a had two copies and b removed only one of them.

8 And (4,NULL) survives everything, because b has no row with a 4 in it at all.

THE TAKEAWAY

Eight rows and the whole module's semantics in one table. Build this fixture once and every set operation becomes checkable by eye.

Testing a set operation

- ◆ **A row appearing twice on one side and once on the other**, which is the only thing separating the plain and ALL forms.
- ◆ **A row containing a null, present on both sides**, which is the null-equality behaviour.
- ◆ **A row present on only one side**, which is the ordinary case and confirms nothing is broken.
- ◆ **And branches with columns in a different order**, which is the only way to see positional matching happen.

THE TAKEAWAY

Four cases, eight rows. The fourth is the one nobody tests, and it is the one that put online revenue against the wrong dates in scenario one.

Why that makes EXCEPT the right diff tool

- ◆ **Two rows that are identical, including their nulls, are the same row.** That is what you mean by no difference.
- ◆ **A join on every column disagrees,** reporting a difference wherever a null sits, on rows that are in fact identical.
- ◆ **So EXCEPT in both directions is the reconciliation,** and it needs no COALESCE and no IS DISTINCT FROM.
- ◆ **Across forty columns that matters enormously,** because the join version needs forty null-safe comparisons written out.

THE TAKEAWAY

This is why the capstone's reconciliation deliverable asks for both directions. EXCEPT one way finds what is missing; the other way finds what is extra.

The both-directions diff

```
1 -- in the old table and not the new: rows that were lost
2 SELECT * FROM old_result
3 EXCEPT
4 SELECT * FROM new_result
5
6 -- in the new and not the old: rows that appeared
7 SELECT * FROM new_result
8 EXCEPT
9 SELECT * FROM old_result
10
11 -- both empty: the two results are identical, nulls included
```

- 11 One direction is not enough. A row that changed value appears in both diffs, and a row that moved appears in neither.

THE TAKEAWAY

Two queries, and between them a complete equality test on whole rows. This is the shape of every regression test in module twenty and the capstone's d09.

Columns match **by position**. The names are decoration, and nothing checks them.

A set operation compares **whole rows for identity**. A join compares **values for equality**. Nulls behave differently in each.

What positional matching does

```
1 SELECT 1 AS p, 'x' AS q
2 UNION ALL
3 SELECT 'y' AS q, 2 AS p
4
5 -- result:
6 --   p      q
7 --   '1'    'x'
8 --   'y'    '2'    <- column p contains 'y'
9
10 -- no error. both columns were widened to VARCHAR.
```

- 8 The aliases said q and p. The positions said first and second. Position won, silently.
- 10 And the type unification widened an integer to text rather than raising, so nothing complained.

THE TAKEAWAY

Measured. Two queries whose aliases are in opposite order, unioned without complaint, producing a column named p that holds values from q.

Why this survives review

- ◆ **The two queries usually look similar**, so a reviewer reading down each one sees plausible column lists and moves on.
- ◆ **Compatible types hide it**. Two integers, or two strings, swap silently and produce entirely believable data.
- ◆ **Incompatible types are the lucky case**, because an error is a gift. Widening to text is the unlucky one.
- ◆ **Name every column explicitly, in the same order, in every branch**, and read them as a column rather than as two lists.

THE TAKEAWAY

The failure mode is worst when the types are most similar. Two date columns swapped by position produce a query that runs for years.

A combined transactions view unions in-store sales and online orders. Both branches select a store key, a date key and a net amount, and both use the same column aliases.

Finance report that online revenue appears against the wrong dates. Investigation shows the online branch selects `date_key` and `store_key` in the opposite order from the store branch. The query has never errored.

What is the cause?

- ☐ A The two branches use different types for the date key, and the implicit cast is corrupting the value.
- ☐ B Set operations match columns by position, not by name, so the aliases were never consulted.
- ☐ C UNION removed rows that differed only by the swapped columns, leaving an inconsistent result.
- ☐ D The column aliases must be identical across branches, and here one branch is missing an alias.

◆ PAUSE AND DECIDE

What is the cause?

- A

A type problem

Both are integers, which is exactly why it runs. A genuine type clash would have raised an error on the first execution.
- B

Position, not name

The first column of each branch lines up regardless of what it is called. Store keys are now in the date column and vice versa.
- C

UNION removed rows

Deduplication removes identical rows. Swapped columns produce different rows, so nothing is removed by it.
- D

Aliases must match

They need not, and here they do match. That is precisely what made it look correct to everybody who read it.

THE SIGNAL

"Both use the same column aliases." That sentence is why nobody caught it. The names line up perfectly and are not what the engine is using.

UNION deduplicates. That is **a sort or a hash** of everything, and it is usually unnecessary.

Measured, on seven million rows

```
1 SELECT sale_key FROM fact_sales_line
2 UNION ALL
3 SELECT sale_key FROM fact_sales_line
4     -> 14,139,472 rows    0.0s
5
6 SELECT sale_key FROM fact_sales_line
7 UNION
8 SELECT sale_key FROM fact_sales_line
9     -> 7,069,736 rows    2.9s
```

9 The same data. The 2.9 seconds bought a deduplication that halved the result, which here is exactly what was wanted.

THE TAKEAWAY

The point is not that UNION is slow. It is that you are paying for a deduplication, and most unions in production are combining sets that cannot overlap.

When UNION is the wrong keyword

- ◆ **Combining partitions.** Last year and this year cannot contain the same row, so there is nothing to deduplicate.
- ◆ **Combining sources.** Store sales and online orders are disjoint by construction, and paying to prove it is waste.
- ◆ **Building a small list of literals,** where you know what is in it because you typed it.
- ◆ **UNION is right when duplicates are genuinely possible** and removing them is the intent, rather than a side effect you tolerated.

THE TAKEAWAY

Use UNION ALL by default and reach for UNION deliberately. The habit most people have is the other way round, and it is silently expensive.

And when UNION is hiding something

- ◆ **If UNION returns fewer rows than UNION ALL**, duplicates existed. That is information, and the keyword threw it away.
- ◆ **So assert the counts are equal**, once, and then use UNION ALL knowing it is safe.
- ◆ **Using UNION to suppress an unexpected duplicate is module four's dedup problem**, without even the ordering.
- ◆ **A duplicate that appears later will be silently absorbed**, and nobody will know a grain has broken.

THE TAKEAWAY

UNION as a safety net is a deduplication with no ORDER BY and no stated key. It removes the symptom of a grain problem and leaves the problem.

A nightly job combines 40 daily partitions into a month table using UNION between each pair. The partitions are written by a load that keys on store, date and transaction, and cannot overlap.

The job takes 26 minutes, most of it after the last partition has been read. A previous engineer left a comment saying UNION is used 'to be safe'.

What is the right change?

- ☐ A Increase the cluster size, since deduplicating 40 partitions is inherently memory-bound.
- ☐ B Union the partitions in pairs, as a balanced tree, so that no single deduplication sees everything.
- ☐ C Assert the partitions are disjoint once, then switch to UNION ALL and keep the assertion as a nightly check.
- ☐ D Replace UNION with INTERSECT, which compares the partitions without producing a combined result.

◆ PAUSE AND DECIDE

What is the right change?

- A

 Bigger cluster
Pays to perform a deduplication that is provably unnecessary. The partitions cannot overlap, so nothing is ever removed.
- B

 Union in pairs
Makes the same unnecessary work slightly cheaper. Every pairing still compares rows that cannot possibly match.
- C

 Assert, then UNION ALL
The deduplication becomes a nightly assertion instead of a nightly cost, and a real duplicate now raises rather than vanishing.
- D

 Use INTERSECT
Answers the opposite question. It would return only rows present in every partition, which for disjoint partitions is nothing.

THE SIGNAL

"To be safe." The comment is honest and the mechanism is backwards: UNION does not detect a duplicate, it conceals one. An assertion detects it.

Precedence. On most engines INTERSECT binds tighter. **On Oracle it does not.**

Whole-row means every column

```
1 -- these two rows are NOT the same row
2 (41, 20260506, 118.40, 'loaded 2026-05-07')
3 (41, 20260506, 118.40, 'loaded 2026-05-08')
4
5 -- so EXCEPT reports a difference that is not a difference
6
7 -- compare the columns that carry meaning
8 SELECT store_key, date_key, net_amount FROM old_result
9 EXCEPT
10 SELECT store_key, date_key, net_amount FROM new_result;
```

- 3 A load timestamp, an audit column, a surrogate key. Any of them makes every row differ.
- 8 Project to the columns you mean before comparing. SELECT * in a diff compares the metadata too.

THE TAKEAWAY

The commonest reason a reconciliation reports 100% differences. EXCEPT is comparing honestly, and one column nobody thought about is in the comparison.

A diff that says what changed

```
1 WITH only_old AS (SELECT * FROM old EXCEPT SELECT * FROM new),
2     only_new AS (SELECT * FROM new EXCEPT SELECT * FROM old)
3 SELECT COALESCE(o.store_key, n.store_key) AS store_key,
4        COALESCE(o.date_key, n.date_key) AS date_key,
5        o.net_amount AS was, n.net_amount AS now,
6        CASE WHEN o.store_key IS NULL THEN 'appeared'
7             WHEN n.store_key IS NULL THEN 'disappeared'
8             ELSE 'changed' END AS verdict
9 FROM only_old o
10 FULL OUTER JOIN only_new n USING (store_key, date_key);
```

10 Join the two diffs on the key. A row in both diffs changed; a row in one appeared or disappeared.

THE TAKEAWAY

EXCEPT tells you that something differs. Joining the two diffs on the key tells you what, and separates a changed value from a missing row.

Why the key join, not another EXCEPT

- ◆ **EXCEPT compares whole rows**, so a changed value looks like one row leaving and a different row arriving.
- ◆ **Joining the diffs on the key reunites them**, and the pair is then visibly one row that changed.
- ◆ **Without that, a 10,000-row diff might be 5,000 changes** or 10,000 genuine additions and removals, and you cannot tell.
- ◆ **Which is a very different conversation** to have with whoever owns the pipeline.

THE TAKEAWAY

Both diffs empty is a pass. When they are not, the number of rows is almost useless and the pairing is what somebody can act on.

Two groupings, two answers

```
1 SELECT * FROM a UNION SELECT * FROM b INTERSECT SELECT * FROM c
2
3 -- grouped as a UNION (b INTERSECT c): 4 rows
4 -- grouped as (a UNION b) INTERSECT c: 1 row
5
6 -- measured: the unparenthesised form gave 4,
7 -- so INTERSECT bound tighter. On Oracle it would give 1.
```

4 Four rows against one, from the same three inputs and the same three keywords. Only the grouping differs.

THE TAKEAWAY

Measured. A four-to-one difference from precedence alone, and no engine warns that the grouping was ambiguous. Parenthesise every mixed chain.

Set operations, by engine

ENGINE	WHAT IT HAS
PostgreSQL, DuckDB	All six. INTERSECT binds tighter, per the standard.
Amazon Redshift	UNION, UNION ALL, INTERSECT, EXCEPT, MINUS. No ALL forms for the latter three.
Oracle 21c+	All six, plus MINUS and MINUS ALL. Equal precedence, left to right.
Oracle before 21c	UNION, UNION ALL, INTERSECT, MINUS only.
SQL Server	UNION, UNION ALL, INTERSECT, EXCEPT. No INTERSECT ALL or EXCEPT ALL.
MySQL 8.0.31+	All six. Before that release, UNION only.

THE TAKEAWAY

Two of the engines in this course lack the ALL forms entirely. Redshift's documentation says so in one sentence, and it is easy to miss until a query fails.

A data quality query combining three result sets with UNION and INTERSECT is ported from PostgreSQL to an Oracle system as part of a partner integration. Both versions run without error.

The Oracle version returns 1 row where the PostgreSQL version returns 4. The three input queries have been checked and return identical rows on both systems.

What is the cause?

- ☐ A Oracle gives all set operators equal precedence and evaluates left to right, where the standard binds INTERSECT tighter.
- ☐ B Oracle's INTERSECT excludes rows containing nulls, where PostgreSQL's treats nulls as equal.
- ☐ C MINUS and EXCEPT have different semantics, and the port substituted one for the other.
- ☐ D The column order differs between the two systems, so the positional matching is producing different rows.

◆ PAUSE AND DECIDE

What is the cause?

A Precedence differs

Oracle's documentation states all set operators have equal precedence, evaluated left to right. The same text groups differently.

C MINUS versus EXCEPT

Oracle's own documentation says *EXCEPT* is a synonym for *MINUS* with exactly the same semantics. And the query uses neither.

B Oracle excludes nulls

It does not. Oracle documents that nulls common to both sides of INTERSECT ALL are returned, at the end of the result.

D Column order

A real hazard and it would change which rows match rather than how the operators group. The inputs are confirmed identical.

THE SIGNAL

"Both versions run without error." A portability defect that produces a different answer rather than a failure is the expensive kind, and precedence is the classic source.

Set operations in a pipeline

- ◆ **UNION ALL to combine sources**, with a literal column naming which source each row came from.
- ◆ **EXCEPT both ways as a regression test**, run against the previous build's output on every deploy.
- ◆ **INTERSECT to find overlap you did not expect**, which should be empty and is a one-line disjointness assertion.
- ◆ **And rarely anything else**. Most other uses are a join written the long way round.

THE TAKEAWAY

The source-literal column is worth the habit. It costs nothing, survives the UNION ALL, and answers 'where did this row come from' forever afterwards.

The disjointness assertion

```
1 -- the assertion that lets you use UNION ALL safely
2 SELECT COUNT(*) AS overlapping_rows FROM (
3     SELECT store_key, date_key, txn_id FROM partition_a
4     INTERSECT
5     SELECT store_key, date_key, txn_id FROM partition_b
6 );
7
8 -- must be zero. If it is, UNION ALL is provably equivalent
9 -- to UNION, and costs nothing.
```

8 Assert on the key rather than the whole row, because two rows can share a key and differ in a measure.

THE TAKEAWAY

Six lines, run once at build time. It converts a nightly deduplication into a nightly check, which is scenario two's answer in one query.

So parenthesise, always

- ◆ **Any chain mixing two different operators needs parentheses**, and it needs them for the reader as much as for the engine.
- ◆ **Relying on precedence is relying on a rule two engines disagree about**, and which neither will warn you about.
- ◆ **A chain of one operator is safe**, because UNION with UNION and EXCEPT with EXCEPT are left-associative everywhere.
- ◆ **And the reader cannot tell what you meant**, which is reason enough on its own.

THE TAKEAWAY

One pair of brackets removes a portability hazard and an ambiguity at the same time. There is no argument against writing them.

ORDER BY belongs to **the whole result**, not to a branch. And the names come from the first branch.

Where the ORDER BY goes

```
1 -- correct: one ordering, at the end
2 SELECT k, v FROM a
3 UNION
4 SELECT k, v FROM b
5 ORDER BY k DESC;
6
7 -- parse error on most engines
8 SELECT k FROM a ORDER BY k
9 UNION
10 SELECT k FROM b;
```

5 This orders the combined result, and it can only reference columns by the first branch's names or by position.

THE TAKEAWAY

If you need a branch ordered for its own sake, it needs a LIMIT too, and then it has to be a subquery. An ordering with nothing to cut is not a promise anyway.

Type unification, and its hazards

- ◆ **The engine finds a common type per position**, widening where it can, so an integer and a numeric become numeric.
- ◆ **Widening to text is the dangerous one**, because almost anything unifies with text and almost nothing then errors.
- ◆ **And precision can be lost quietly**, where a numeric and a float unify to float and the pennies stop being exact.
- ◆ **Cast explicitly in every branch** when the types are not already identical, so the unification has nothing to decide.

THE TAKEAWAY

Redshift will not implicitly convert across type categories and raises instead, which is stricter and kinder. Engines that widen to text are the ones to watch.

UNION ALL is **concatenation**. The other five are **comparisons**, and comparisons cost.

A reconciliation compares a rebuilt fact table against the original, using a FULL OUTER JOIN on all 34 columns and counting rows where any column differs. It reports 118,000 differences.

Spot checks show the flagged rows are identical in every visible respect. All of them contain at least one null. The tables have 7 million rows each.

What should they use instead?

- ☐ A COALESCE every column to a sentinel before joining, so that nulls compare equal.
- ☐ B Accept the result. Rows containing nulls cannot be proven identical, so flagging them is correct.
- ☐ C Rewrite the join conditions to use IS NOT DISTINCT FROM on all 34 columns.
- ☐ D EXCEPT in both directions, where whole-row comparison treats nulls as equal and needs no per-column handling.

◆ PAUSE AND DECIDE

What should they use instead?

- A

 COALESCE to a sentinel
Works until a real value equals the sentinel, and that has to be chosen per type across 34 columns. It is 34 chances to pick badly.
- B

 Accept it
Then no reconciliation can ever pass on a table containing nulls, which is every table. The tool is wrong, not the requirement.
- C

 IS NOT DISTINCT FROM
Correct, and it is 34 predicates written by hand, and a new column silently falls outside the comparison.
- D

 EXCEPT both ways
Whole-row comparison with null equality, in two queries, and a new column is included automatically.

THE SIGNAL

"All of them contain at least one null." That is the entire diagnosis. A join compares and cannot tell; a set operation collects and can.

Choosing, in four questions

01

Can they overlap?

If not, UNION ALL. Assert disjointness once rather than paying for it nightly.

02

Do duplicates matter?

If the count of a row is meaningful, you want an ALL form, and two engines do not have them.

03

Mixed operators?

Parenthesise. Oracle and the standard disagree, and neither will warn you.

04

Comparing tables?

EXCEPT both ways. It is the only null-equal whole-row comparison SQL gives you.

THE TAKEAWAY

Question four is the one worth taking away. Every other null-handling tool in SQL needs writing per column, and this one does not.

What Part 6 is about

- ◆ **This module: combining results**, several queries into one, with rules about nulls and duplicates.
- ◆ **Module nineteen: multiplying results**, one query into several levels of aggregation, in one pass.
- ◆ **They meet at the subtotal**. Before GROUPING SETS, a subtotalled report was a UNION of three queries.
- ◆ **And module nineteen is that, done properly**, in one scan instead of three.

THE TAKEAWAY

The union-of-three-queries subtotal report is still everywhere in inherited code. Module nineteen is what it should have been, and reads it for you.

The subtotal report, the old way

```
1 SELECT region_key, store_format, SUM(net_sales) FROM t
2   GROUP BY region_key, store_format
3 UNION ALL
4 SELECT region_key, NULL, SUM(net_sales) FROM t
5   GROUP BY region_key
6 UNION ALL
7 SELECT NULL, NULL, SUM(net_sales) FROM t;
```

- 4 A literal NULL standing in for 'all formats', which is indistinguishable from a real null in the data.
- 7 And three scans of the same table, to produce three levels of one aggregation.

THE TAKEAWAY

Correct, slow, and ambiguous about its nulls. Module nineteen replaces all seven lines with one clause and adds a function that tells the two kinds of null apart.

Recognising it in inherited code

- ◆ **A UNION ALL of the same query with different GROUP BY clauses.** That is a subtotal report, written the long way.
- ◆ **A UNION used where the inputs cannot overlap.** Somebody was being careful, and paid for it nightly.
- ◆ **A FULL OUTER JOIN on every column.** That is a diff, and it is wrong on every row containing a null.
- ◆ **All three have a shorter, faster, more correct form,** and all three are extremely common.

THE TAKEAWAY

Three patterns, three replacements. The third is the one that has been quietly reporting false differences, and the one nobody has questioned.

The confusions, settled

BELIEF	THE TRUTH
Columns line up by name	By position. The names come from the first branch.
NULL never equals NULL	It does here. Set operations collect rather than compare.
UNION is the safe default	It is a deduplication you are paying for, and usually hiding something.
INTERSECT ALL exists everywhere	Not on Redshift or SQL Server. Oracle gained it in 21c.
Precedence is standard	Oracle evaluates left to right; the standard binds INTERSECT tighter.
EXCEPT one way is a diff	Two directions, or you miss the rows that appeared.

THE TAKEAWAY

Six beliefs, and five of them are held by people who use these operators daily. The syntax is so simple that nobody goes back and reads the rules.

Five habits

- ◆ **UNION ALL by default**, with an assertion that the inputs are disjoint, rather than UNION as a reflex.
- ◆ **Name every column in every branch, in the same order**, and read them as a column rather than as two lists.
- ◆ **Parenthesise any chain that mixes operators**, because precedence is a portability hazard and an ambiguity.
- ◆ **Cast explicitly when types differ**, so that unification has nothing to decide and nothing to widen.
- ◆ **Use EXCEPT both ways to compare results**, and treat both being empty as the definition of identical.

THE TAKEAWAY

The last one is the habit that earns its place beyond this module. It is the comparison every regression test in Part 7 is built on.

Set operations combine results. **Module nineteen** produces several at once.